DENA: A Preliminary Distributed Expert Neural Architecture for Modular Expert Routing and Feedback-Driven Learning

Harsha Sai Reddy Eada
Independent Researcher
harshasaireddyind@gmail.com

Draft version 1.0 – April 2026

Abstract

Modern artificial intelligence systems are often implemented as dense models, mixture-of-experts models, retrieval-augmented systems, tool-using agents, or multi-agent frameworks. These approaches have achieved strong performance, but they do not always provide a unified architecture for independently trained, replaceable, distributed expert subsystems with explicit routing, aggregation, verification, memory, and feedback-driven modular updates. This paper proposes DENA, the Distributed Expert Neural Architecture, as a preliminary architecture for modular expert AI systems. In DENA, inputs are transformed into task representations, decomposed into subtasks, routed to one or more independently trained expert subsystems, aggregated into a final answer, verified for acceptance or rerouting, and stored in memory for later feedback-based improvement. We provide a formal definition, mathematical formulation, prototype implementation, and a controlled toy benchmark called DENA-Bench v0. Preliminary results show that sparse DENA routing can approach all-expert ensemble quality in the toy benchmark while using lower estimated expert activation cost. These results do not establish superiority over production dense or MoE models, but they demonstrate that DENA is implementable, measurable, interpretable, and suitable for further research in modular expert AI architectures.

Keywords: modular AI, expert routing, mixture of experts, model orchestration, distributed AI systems, feedback learning, AI architecture

Contents

1 Introduction **2**
1.1 Contributions . . . 2
1.2 Scope of the paper . . . 2

2 Background and Related Work **3**

3 DENA Architecture **4**
3.1 Core definition . . . 4
3.2 Main components . . . 5
3.3 Minimum criteria . . . 5
3.4 Training and update cycle . . . 5

4 Mathematical Formulation **6**
4.1 System definition . . . 6
4.2 Input representation and task decomposition . . . 6
4.3 Expert routing . . . 7
4.4 Expert output and aggregation . . . 7
4.5 Verification and memory . . . 8
4.6 Training objective . . . 8

5 Prototype Implementation **8**
5.1 Overview . . . 8
5.2 Prototype pseudocode . . . 9
5.3 Prototype limitations . . . 9

6 DENA-Bench v0 **9**
6.1 Benchmark purpose . . . 9
6.2 Systems compared . . . 10
6.3 Metrics . . . 10

7 Experimental Results **11**
7.1 Main benchmark table . . . 11
7.2 Result interpretation . . . 11
7.3 What the results do and do not show . . . 12

8 Comparison with Existing Methods **13**

9 Limitations **13**

10 Safety and Ethical Considerations **14**

11 Future Work **14**

12 Conclusion **15**

A Reproducibility Notes **15**

B Suggested Citation **15**

1 Introduction

Large neural models have become a dominant paradigm in artificial intelligence. Dense transformer models process inputs through a single parameter space and have shown strong generalization across language, vision, and multimodal tasks. However, dense models can be expensive to train and update, difficult to interpret internally, and vulnerable to interference when new domain knowledge is added through fine-tuning.

Mixture-of-Experts (MoE) architectures improve efficiency by activating a sparse subset of internal expert subnetworks. Ensembles combine multiple models to improve accuracy. Retrieval-Augmented Generation (RAG) systems connect a generator to external knowledge. Tool-using agents call calculators, search engines, code interpreters, or APIs. Multi-agent systems coordinate multiple language-model agents. Each of these approaches contributes useful ideas, but none of them alone fully captures the architectural pattern of independently trained, replaceable, distributed expert subsystems with explicit routing, aggregation, verification, memory, and feedback-driven updates.

This paper introduces DENA, the *Distributed Expert Neural Architecture*. DENA is proposed as a modular expert AI architecture where multiple specialized experts are trained or maintained independently and coordinated through a central orchestration framework. For each input, DENA decomposes the task, selects appropriate experts, collects their outputs, aggregates the results, verifies the final response, and stores feedback for system improvement.

1.1 Contributions

This paper makes the following preliminary contributions:

1. It proposes DENA as a distributed expert architecture for modular expert routing and feedback-driven learning.
2. It provides a formal definition of DENA and minimum criteria for a system to qualify as a DENA-style architecture.
3. It presents a mathematical formulation of expert selection, routing probability, aggregation, verification, memory, and update objectives.
4. It describes a minimal prototype implementation with rule-based experts, heuristic routing, weighted aggregation, verification, and JSONL memory.
5. It introduces DENA-Bench v0, a controlled toy benchmark for evaluating routing, answer relevance, exact correctness, estimated cost, and estimated latency.
6. It compares DENA conceptually with dense models, MoE, ensembles, RAG, tool-using agents, model orchestration systems, multi-agent systems, federated learning, and knowledge distillation.

1.2 Scope of the paper

This is an early-stage architecture paper. The current prototype uses toy experts and controlled benchmark tasks. The results should be interpreted as architectural validation, not as evidence that DENA outperforms production dense transformers or true MoE systems. The main goal is to formalize the architecture and create a reproducible foundation for future research with neural experts and stronger benchmarks.

2 Background and Related Work

Dense transformer models. The transformer architecture introduced attention-based sequence modeling without recurrence, enabling large-scale parallel training and becoming the foundation for many modern language models [1]. Dense models commonly use most parameters for most inputs, which can simplify inference but reduce modular updateability.

Mixture of Experts. Mixture-of-Experts methods use multiple expert components and a gating mechanism to select experts. Early work on adaptive mixtures of local experts introduced conditional computation with specialized expert modules [2]. Sparsely gated MoE layers demonstrated the scalability of large expert models with sparse activation [3]. Switch Transformers simplified token routing to one expert and showed that sparse expert models can scale efficiently [4]. DENA is related to MoE but treats experts as independently trainable and potentially distributed subsystems rather than only internal neural layers.

Ensemble learning and distillation. Ensembles combine outputs from multiple models, often improving accuracy but increasing inference cost. Knowledge distillation transfers knowledge from large models or ensembles into smaller deployable models [5]. DENA differs from standard ensembles by dynamically selecting experts instead of executing every model, and it can also act as a teacher system for later distillation.

Retrieval-Augmented Generation. RAG combines a parametric generator with non-parametric memory, typically retrieved documents from an external index [6]. DENA can include RAG modules as experts, but the architecture generalizes beyond document retrieval to heterogeneous expert systems.

Tool-using agents and model orchestration. Toolformer learns to call external tools during language modeling [7]. ReAct combines reasoning traces with environment actions [8]. HuggingGPT uses a language model as a controller to plan tasks, select models, execute them, and summarize results [9]. These systems are related to DENA because they involve external model or tool selection. DENA attempts to formalize this pattern as a trainable distributed expert architecture with explicit metadata, routing objectives, aggregation objectives, verification, memory, and modular updates.

Multi-agent systems. Multi-agent LLM frameworks coordinate multiple agents that can converse, use tools, and collaborate. AutoGen is one example of a framework for multi-agent LLM applications [10]. DENA differs by treating the expert subsystem as the primary unit rather than the conversational agent; an expert can be a neural model, retriever, symbolic tool, verifier, human-reviewed module, or external service.

Federated and feedback learning. Federated learning trains models across decentralized data sources without centralizing all data [11]. Human preference learning and reinforcement learning from feedback have shown that human judgments can improve model behavior [12, 13]. DENA can incorporate these ideas by training experts locally, collecting feedback, and updating routers, aggregators, verifiers, or selected experts.

3 DENA Architecture

3.1 Core definition

DENA is a modular artificial intelligence architecture composed of multiple independently trained expert subsystems connected through a central orchestration framework. For each input task, DENA analyzes the task, dynamically selects one or more suitable experts, gathers their outputs, aggregates and verifies the results, and uses feedback to improve expert performance, routing, and system-level decision-making over time.

In simple form:

```
Input -> Orchestrator -> Router -> Expert Models -> Aggregator -> Verifier ->
Output
```

with memory and feedback connected to routing, expert selection, and modular updates.

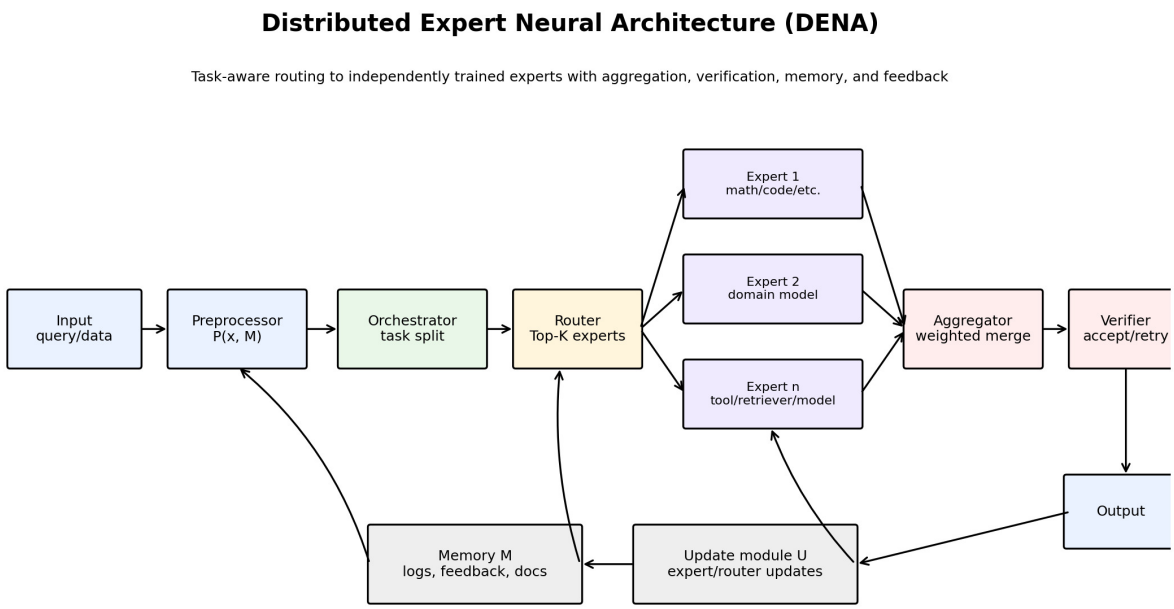

Figure 1: High-level architecture of DENA. Inputs are decomposed into subtasks, routed to independently trained experts, aggregated, verified, and stored in memory for feedback-driven improvement.

3.2 Main components

Component	Role
Input layer	Accepts a query, document, image, audio signal, data point, or task description.
Preprocessor	Converts raw input into tokens, embeddings, metadata, or task representations.
Orchestrator	Decomposes complex inputs into subtasks and controls the execution plan.
Router	Selects one or more experts based on task relevance, quality, cost, latency, memory, and feedback.
Expert models	Independently trained specialist subsystems that solve domain-specific subtasks.
Aggregator	Combines selected expert outputs using weights, confidence, voting, synthesis, or learned aggregation.
Verifier	Scores the final answer and can accept, reject, reroute, or request human review.
Memory	Stores past inputs, expert selections, outputs, feedback, routing history, and domain knowledge.
Update module	Updates experts, router, aggregator, verifier, or memory from feedback or benchmark results.

Table 1: Core components of a DENA-style system.

3.3 Minimum criteria

A system should satisfy the following conditions to qualify as a DENA-style architecture:

1. It contains at least two expert subsystems.
2. Experts have defined specializations, such as domain, modality, task type, or reasoning type.
3. Expert selection is dynamic and depends on the input or subtask.
4. Outputs from selected experts are aggregated, selected, verified, or synthesized.
5. At least one expert can be updated or replaced without retraining the whole architecture.
6. The system supports feedback-based improvement of experts, routing, aggregation, verification, or memory.

3.4 Training and update cycle

Unlike a monolithic training process, DENA allows experts to be trained independently, registered with metadata, and updated modularly. A benchmark or feedback loop can identify weak experts or routing failures and update only the relevant module.

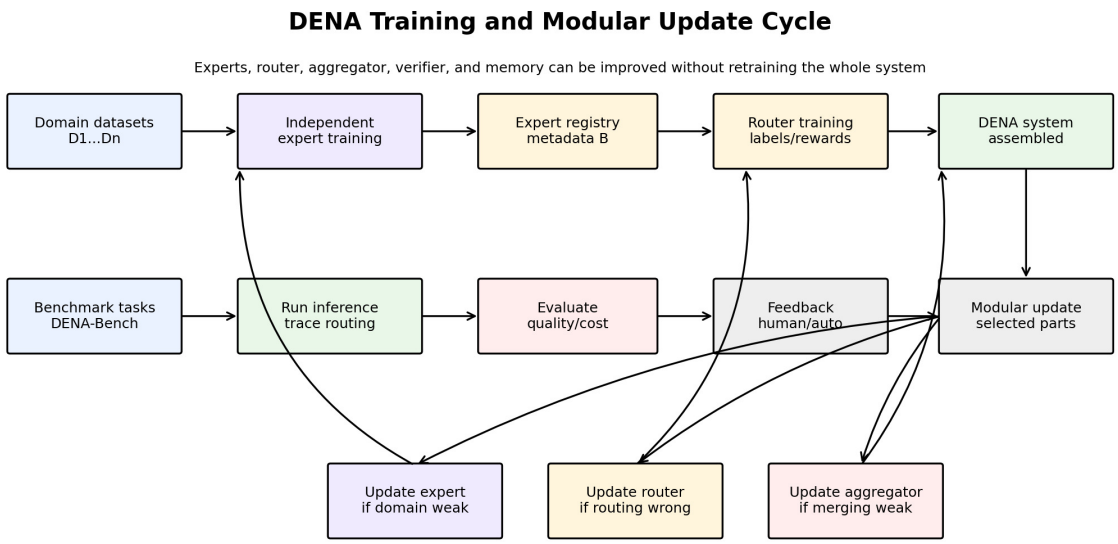

Figure 2: DENA training and modular update cycle. Experts, router, aggregator, verifier, and memory can be improved without retraining the entire system.

4 Mathematical Formulation

4.1 System definition

A DENA system is defined as:

$$\mathcal{DENA} = (\mathcal{X}, \mathcal{Y}, \mathcal{E}, O, R, A, V, M, F, U), \tag{1}$$

where $\mathcal{X}$ is the input space, $\mathcal{Y}$ is the output space, $\mathcal{E}$ is the set of experts, O is the orchestrator, R is the router, A is the aggregator, V is the verifier, M is memory, F is feedback, and U is the update mechanism.

The expert set is:

$$\mathcal{E} = \{E_1, E_2, \ldots, E_n\}. \tag{2}$$

Each expert can be represented as:

$$E_i = (\theta_i, D_i, \mathcal{T}_i, c_i, \kappa_i, \lambda_i), \tag{3}$$

where θ_i are model parameters, D_i is the expert dataset, $\mathcal{T}_i$ is the task or domain specialization, c_i is a confidence function, κ_i is computational cost, and λ_i is latency.

4.2 Input representation and task decomposition

For an input $x_t \in \mathcal{X}$, the preprocessor creates a task representation:

$$z_t = P(x_t, M_t), \tag{4}$$

where M_t is memory at time t.

The orchestrator decomposes the task into subtasks:

$$T_t = O_\omega(z_t) = \{\tau_1, \tau_2, \ldots, \tau_m\}. \tag{5}$$

4.3 Expert routing

For subtask τ_j, the router scores every expert:

$$s_{ij} = g_\psi(\tau_j, b_i, M_t), \tag{6}$$

where b_i is expert metadata and g_ψ is a parameterized routing function. The score is converted to a routing probability:

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^{n} \exp(s_{kj})}. \tag{7}$$

The selected expert set is:

$$S_j = \mathrm{TopK}(\{p_{1j}, p_{2j}, \ldots, p_{nj}\}, K). \tag{8}$$

Cost-aware routing can use a modified score:

$$\hat{s}_{ij} = s_{ij} - \beta\kappa_i - \gamma\lambda_i + \delta q_i, \tag{9}$$

where q_i is expert quality, β penalizes cost, γ penalizes latency, and δ rewards quality.

4.4 Expert output and aggregation

Each selected expert produces an output and confidence score:

$$(o_{ij}, u_{ij}) = E_i(\tau_j; \theta_i), \quad i \in S_j. \tag{10}$$

The aggregation weight can be defined as:

$$w_{ij} = \mathrm{softmax}\left(\eta_1 p_{ij} + \eta_2 u_{ij} + \eta_3 q_i - \eta_4 \kappa_i - \eta_5 \lambda_i\right). \tag{11}$$

The subtask result is:

$$r_j = A_\alpha(\{(o_{ij}, w_{ij}, u_{ij}) : i \in S_j\}, \tau_j). \tag{12}$$

The global output is:

$$y_t = A_\alpha^{global}(\{r_1, r_2, \ldots, r_m\}, z_t, M_t). \tag{13}$$

Equivalently, the compact inference equation is:

$$\boxed{y_t = A_\alpha^{global}\left(\left\{A_\alpha\left(\{E_i(\tau_j;\theta_i), w_{ij}\}_{i\in S_j}\right)\right\}_{j=1}^{m}, M_t\right).} \tag{14}$$

4.5 Verification and memory

A verifier scores the final output:

$$v_t = V_\nu(x_t, y_t, \{o_{ij}\}), \quad v_t \in [0, 1]. \tag{15}$$

The answer is accepted when:

$$Accept(y_t) = \begin{cases} 1, & V_\nu(x_t, y_t, \{o_{ij}\}) \geq \rho, \\ 0, & \text{otherwise}, \end{cases} \tag{16}$$

where ρ is an acceptance threshold.
Memory is updated as:

$$M_{t+1} = M_t \cup \{x_t, T_t, S_t, O_t, y_t, v_t, h_t\}, \tag{17}$$

where h_t is feedback.

4.6 Training objective

The complete system objective can be written as:

$$\mathcal{L}_{DENA} = \mathcal{L}_{task} + \lambda_1 \mathcal{L}_{router} + \lambda_2 \mathcal{L}_{agg} + \lambda_3 \mathcal{L}_{verify} + \lambda_4 \mathcal{L}_{calib} + \lambda_5 C + \lambda_6 L + \lambda_7 Risk. \tag{18}$$

Here C is compute cost, L is latency, and $Risk$ is a penalty for unsafe or unreliable outputs. A DENA-specific optimization separates expert training from system-level training:

$$\min_{\psi,\alpha,\nu} \mathcal{L}_{system} \quad \text{while independently optimizing} \quad \min_{\theta_i} \mathcal{L}^{(i)}_{expert}. \tag{19}$$

A modular update of expert E_k changes only θ_k:

$$\theta_k \leftarrow \theta_k - \eta_k \nabla_{\theta_k} \mathcal{L}^{(k)}_{expert}, \quad \Delta\theta_i = 0 \quad \forall i \neq k. \tag{20}$$

5 Prototype Implementation

5.1 Overview

The DENA v0 prototype is a dependency-free Python implementation designed to validate the architecture rather than maximize task performance. It implements:

- a preprocessor with tokenization and memory retrieval,
- an orchestrator that decomposes inputs into domain-tagged subtasks,
- a router that scores experts using domain match, keyword relevance, quality, cost, latency, and memory bonus,
- five rule-based experts,
- a weighted aggregator,
- a simple verifier,

- JSONL memory and optional feedback storage.

The current experts are:

Expert	Purpose
Expert_Math	Arithmetic and simple mathematical tasks
Expert_Code	Programming, debugging, and algorithm planning
Expert_Research	Architecture, benchmark, and research-related reasoning
Expert_Definition	Definitions of DENA, MoE, dense models, routers, and aggregators
Expert_General	General fallback expert

5.2 Prototype pseudocode

Listing 1: Simplified DENA v0 inference flow.

```
def process(query):
    rep = preprocessor.transform(query)
    subtasks = orchestrator.decompose(rep)
    subtask_results = []

    for subtask in subtasks:
        selected = router.route(subtask, rep, top_k=K)
        outputs = [expert.run(subtask) for expert in selected]
        result = aggregator.aggregate_subtask(subtask, selected, outputs)
        subtask_results.append(result)

    final_answer = aggregator.aggregate_global(query, subtask_results)
    verification_score, accepted = verifier.verify(final_answer, subtask_results)
    memory.append(query, subtasks, selected, final_answer, verification_score)
    return final_answer
```

5.3 Prototype limitations

DENA v0 uses rule-based experts, heuristic routing, formula-based aggregation, simple verification, and JSONL memory. These choices make the system transparent and easy to test, but they do not represent the full potential of neural DENA systems. The next prototype should replace rule-based experts with neural experts, train the router, train the aggregator, use vector memory, and evaluate on larger public benchmarks.

6 DENA-Bench v0

6.1 Benchmark purpose

DENA-Bench v0 is a controlled toy benchmark for evaluating whether the architecture can be measured. It tests routing correctness, expert coverage, answer relevance, exact-answer correctness, estimated expert activation cost, estimated latency, and interpretability.

The benchmark contains five categories:

- **math_exact**: arithmetic tasks with exact expected answers,

- **code_debug**: programming and debugging tasks,
- **definition**: definitions of DENA-related concepts,
- **architecture_research**: architecture and benchmark design questions,
- **mixed_domain**: questions requiring more than one expert type.

6.2 Systems compared

System	Description
DENA_top1	DENA with top-1 expert selection per subtask.
DENA_top2	DENA with top-2 expert selection per subtask.
DENA_top3	DENA with top-3 expert selection per subtask.
AllExperts_proxy	Executes all experts for every task, acting as an ensemble-style proxy.
GeneralOnly_proxy	Uses only the general expert, acting as a dense/general-model proxy.
RandomRouter_proxy	Selects experts randomly.
OracleRouter_upper_bound	Uses benchmark labels to select expected experts; not deployable, only an upper-bound reference.

Table 2: Benchmark systems evaluated in DENA-Bench v0.

6.3 Metrics

Routing precision, recall, and F1 are defined as:

$$Precision = \frac{|\hat{S} \cap S^*|}{|\hat{S}|}, \quad Recall = \frac{|\hat{S} \cap S^*|}{|S^*|}, \quad F1 = \frac{2PR}{P+R}, \tag{21}$$

where $\hat{S}$ is the selected expert set and S^* is the expected expert set.
Keyword score is:

$$KeywordScore = \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}[keyword_k \in y]. \tag{22}$$

For exact-answer tasks:

$$ExactMatch = \mathbf{1}[\text{gold answer appears in output}]. \tag{23}$$

The benchmark defines answer score as:

$$AnswerScore = \begin{cases} 0.7 \cdot ExactMatch + 0.3 \cdot KeywordScore, & \text{exact task,} \\ KeywordScore, & \text{otherwise.} \end{cases} \tag{24}$$

Quality score is:

$$Quality = 0.45 \cdot RoutingRecall + 0.45 \cdot AnswerScore + 0.10 \cdot Accepted. \tag{25}$$

Cost-adjusted score is:

$$CostAdjusted = Quality - 0.10 \cdot Cost - 0.05 \cdot Latency. \tag{26}$$

These metrics are intentionally simple for DENA-Bench v0. Future versions should use real token cost, FLOPs, wall-clock latency, memory usage, and human evaluation.

7 Experimental Results

7.1 Main benchmark table

System	Quality	Cost-Adj.	Routing F1	Answer	Avg Cost	Avg Latency
DENA_top1	0.961	0.932	0.903	0.939	0.194	0.194
DENA_top2	0.974	0.928	0.741	0.949	0.337	0.243
DENA_top3	0.974	0.911	0.602	0.949	0.485	0.300
OracleRouter_upper_bound	0.950	0.908	0.733	0.929	0.309	0.227
AllExperts_proxy	0.977	0.883	0.420	0.949	0.788	0.315
RandomRouter_proxy	0.741	0.701	0.432	0.803	0.299	0.213
GeneralOnly_proxy	0.302	0.293	0.000	0.449	0.063	0.063

Table 3: DENA-Bench v0 results. The benchmark is toy-scale and intended for architectural validation.

7.2 Result interpretation

DENA_top1 achieved the best cost-adjusted score in the toy benchmark. DENA_top2 nearly matched AllExperts_proxy in quality while using much lower estimated expert activation cost. AllExperts_proxy obtained the highest quality score but selected unnecessary experts for most queries, causing low routing F1 and high cost. GeneralOnly_proxy was cheapest but had the weakest routing and answer scores. RandomRouter_proxy performed worse than DENA variants, indicating that the heuristic routing mechanism is meaningful in the current controlled setting.

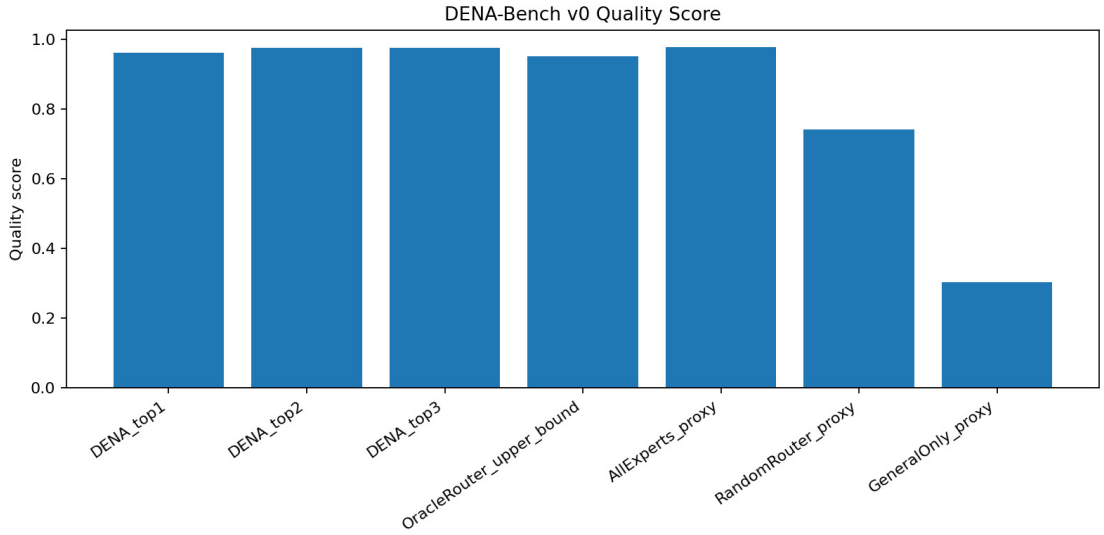

Figure 3: Quality score comparison in DENA-Bench v0.

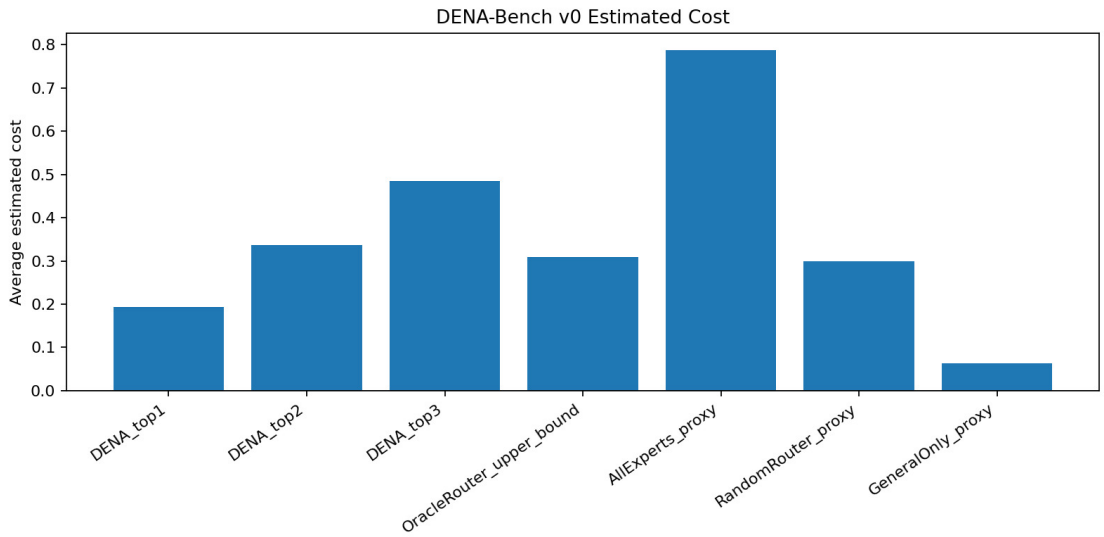

Figure 4: Estimated cost comparison in DENA-Bench v0.

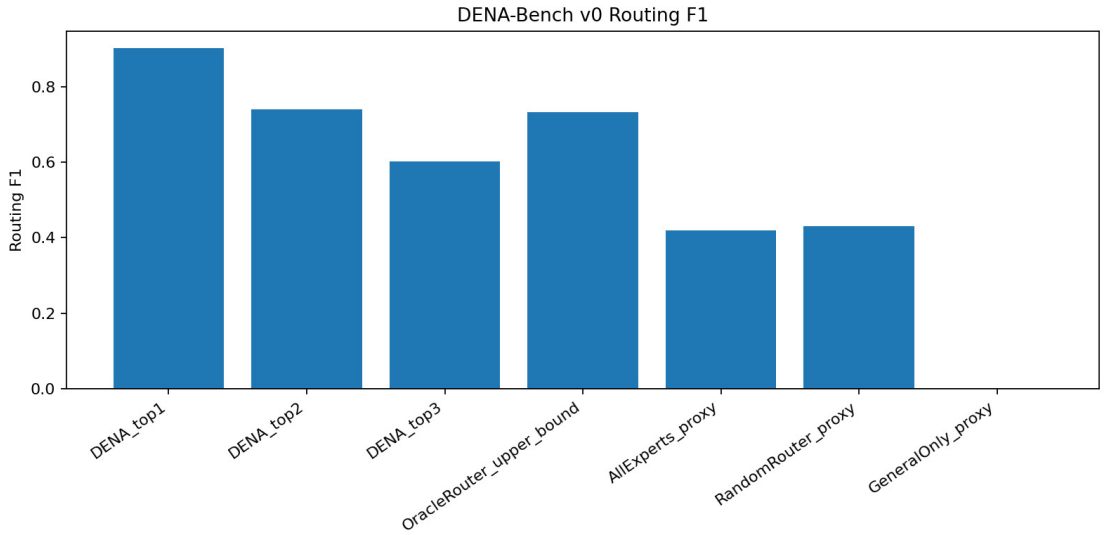

Figure 5: Routing F1 comparison in DENA-Bench v0.

7.3 What the results do and do not show

The benchmark supports the claim that DENA is implementable, measurable, interpretable, and capable of cost-aware expert routing in a controlled toy setting. It does not prove that DENA outperforms production dense transformers, true MoE systems, large-scale RAG pipelines, or mature multi-agent frameworks. Stronger claims require neural experts, larger datasets, real hardware measurements, and stronger baselines.

8 Comparison with Existing Methods

Method	Core idea	Strength	DENA difference
Dense model	One large model handles most tasks	Simple inference and broad generalization	DENA distributes intelligence across experts and supports modular updates.
MoE	Internal experts selected by a gate	Sparse activation and scalable capacity	DENA experts can be external, heterogeneous, independently trained, and replaceable.
Ensemble	Multiple models combined	Often improves accuracy	DENA dynamically selects only needed experts to reduce cost.
RAG	Retrieve documents for generation	Strong for factual grounding	DENA can use RAG as one expert type but generalizes to many expert systems.
Tool agent	LLM calls APIs or tools	Useful for external actions	DENA treats tools as possible experts within a broader routing, aggregation, verification, and update architecture.
Multi-agent system	Agents collaborate or debate	Flexible and creative	DENA uses structured expert routing rather than unconstrained agent conversation.
Federated learning	Train across decentralized data	Privacy-aware training	DENA distributes expert responsibility and can incorporate federated experts.
Distillation	Compress knowledge into student model	Efficient deployment	DENA preserves modular experts and can later distill them into a student.

Table 4: Conceptual comparison between DENA and related methods.

The cleanest positioning is that DENA generalizes the expert-selection idea beyond traditional internal MoE by treating experts as independently trained, distributed, replaceable subsystems. It also differs from model orchestration systems by explicitly formalizing routing objectives, aggregation objectives, verification, metadata, memory, and feedback-based updates.

9 Limitations

The current work has several limitations:

- The prototype experts are rule-based rather than neural experts.
- The router is heuristic rather than trained on large routing data.
- The aggregator is formula-based rather than learned from preference or task data.
- The verifier is simple and not reliable enough for high-stakes applications.

- DENA-Bench v0 is small and toy-scale.
- The compared baselines are proxies, not production-grade dense, MoE, RAG, or agent systems.
- Cost and latency are estimated from expert metadata, not measured on real hardware.
- The current implementation does not yet include multimodal experts, external APIs, vector databases, or neural feedback learning.

These limitations are intentional for the preliminary stage. The purpose of DENA v0 is to define and validate the architecture skeleton before scaling.

10 Safety and Ethical Considerations

DENA-style systems may be useful in education, enterprise AI, scientific research, cybersecurity, healthcare support, legal analysis, and defense decision support. However, modular expert systems also create risks. A bad router may select the wrong expert; an overconfident expert may produce unreliable output; an aggregator may merge conflicting results; and a distributed architecture may increase the attack surface. High-stakes uses require human oversight, audit logs, safety filters, licensing review, privacy protection, and strong verification.

In particular, DENA should not be deployed for medical, legal, financial, or defense decisions without expert human review and domain-specific validation. Future DENA systems should include calibrated confidence, expert provenance, verifier-triggered rejection, red-team testing, and safe fallback behavior.

11 Future Work

Future work should focus on:

1. replacing rule-based experts with neural experts,
2. training a router on labeled routing data,
3. training the aggregator using supervised targets or preference data,
4. using vector memory and retrieval-based expert context,
5. evaluating on public benchmark datasets,
6. comparing against true dense, MoE, RAG, ensemble, and agent baselines,
7. measuring real latency, token usage, FLOPs, memory, and monetary cost,
8. testing modular update efficiency and failure recovery,
9. adding human feedback loops for lifelong improvement,
10. exploring DENA-to-student distillation and federated DENA experts.

12 Conclusion

This paper introduced DENA, a preliminary Distributed Expert Neural Architecture for modular expert routing, aggregation, verification, memory, and feedback-driven learning. We defined the architecture formally, provided a mathematical formulation, implemented a minimal prototype, and introduced DENA-Bench v0 for controlled evaluation. Preliminary results show promising cost-aware routing behavior in a toy benchmark, especially when DENA variants are compared with all-expert, random-router, and general-only proxy baselines. The current work is early-stage and should be interpreted as an architectural foundation rather than a final performance claim. Future research will scale DENA to neural experts, trainable routing, larger benchmarks, and real-world comparisons.

A Reproducibility Notes

The prototype and benchmark can be run with Python. In the current draft package, the main commands are:

```
python dena_prototype.py --demo
python run_dena_experiments.py
python run_dena_benchmarks.py
```

The public release should include:

- source code for DENA v0,
- DENA-Bench v0 dataset,
- experiment runner,
- benchmark runner,
- result CSV files,
- architecture diagrams,
- this paper PDF,
- citation metadata.

B Suggested Citation

After public release, the work can be cited as:

```
@misc{eada2026dena,
  title={DENA: A Preliminary Distributed Expert Neural Architecture
        for Modular Expert Routing and Feedback-Driven Learning},
  author={Harsha Sai Reddy Eada},
  year={2026},
  note={Preprint}
}
```

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[2] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[4] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[5] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. In *NeurIPS Deep Learning and Representation Learning Workshop*, 2015.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[9] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. arXiv preprint arXiv:2303.17580, 2023.

[10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

[11] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[12] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.

[13] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.